



Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impossible for Chess because it would need a table containing an action for every possible percept sequence. In Chess, the number of possible game states and move sequences is extremely large. As the agent's lifetime increases, the number of possible percept sequences grows exponentially, causing a combinatorial explosion. Therefore, the table becomes too large to store and use in practice.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
class SimpleReflexAgent:
    def sense_and_act(self, percept):
        if percept['food_here']:
            return 'Right'
        elif percept['wall_ahead']:
            return 'Up'
        else:
            return 'Right'
```

The if, elif, and else statements implement the condition-action rules. Each checks the current percept and immediately selects an action. For example, if `percept['food_here']` is the condition and `return 'Right'` is the corresponding action. This makes the agent a Simple Reflex Agent because it chooses actions based only on the current percept, without using memory.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The `SimpleReflexAgent` got stuck because it operates under partial observability and has no percept history. It only considers the current percept, such as `wall_ahead` or `food_here`, and does not remember previous states or actions. Therefore, when it encounters the same percept again, it chooses the same action again. This can cause it to repeatedly visit the same cells and enter an infinite loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The `ModelBasedAgent` uses a transition model to predict how its position changes after an action. The `get_next_position()` function predicts the next cell, while `update_position()` updates the internal state when movement succeeds. The sensor model is represented by `get_percept()`, which provides limited information such as whether there is a wall or food ahead. When an action fails, the agent records the target in `blocked_cells`, allowing it to use previous experience when selecting future actions. Thus, the agent combines current percepts with its internal state to make decisions.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING